

CPU minimizes latency. GPU maximizes throughput.

Both are parallel—but a few sophisticated CPU cores finish diverse tasks quickly, while many GPU lanes move far more total work.

CPU

LATENCY FIRST



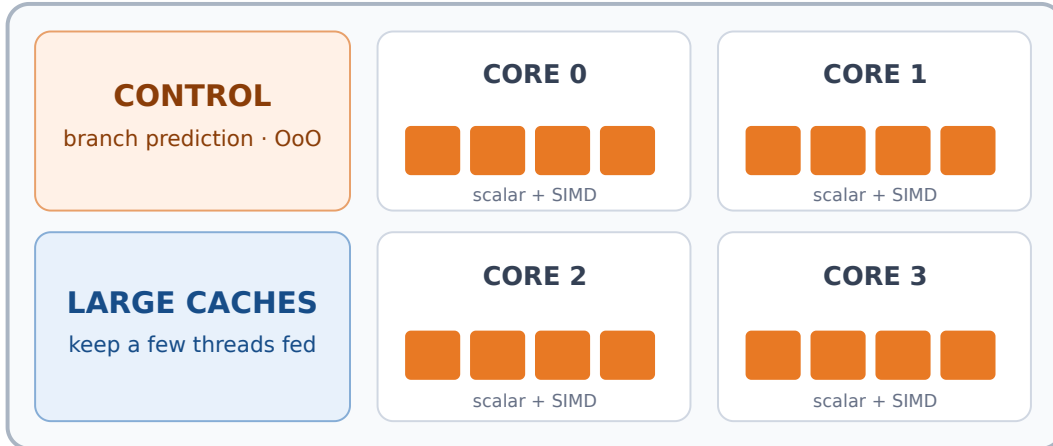
FEW PASSENGERS

Sports car

A few passengers reach different destinations quickly.

SIMPLIFIED SILICON LAYOUT

CONCEPTUAL · NOT TO SCALE



few large, flexible execution engines

GPU

THROUGHPUT FIRST



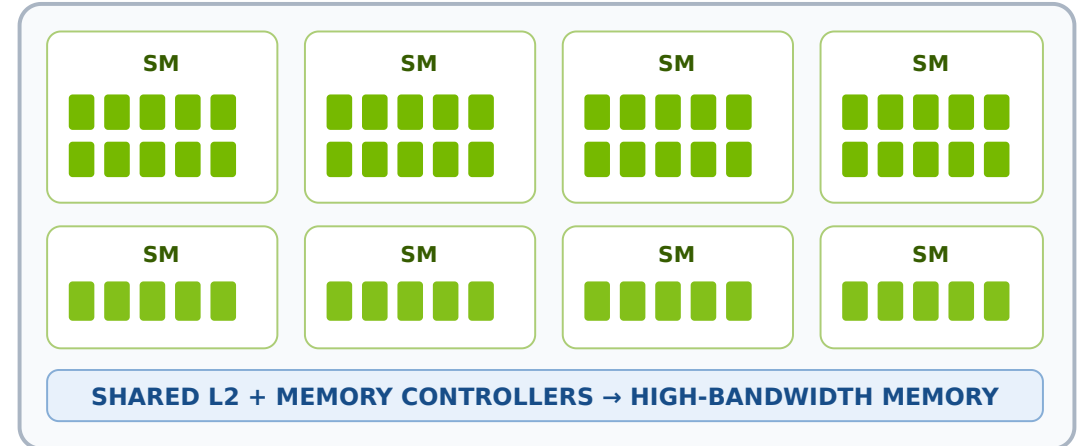
MANY PASSENGERS

Bus

Many passengers following similar routes move together.

SIMPLIFIED SILICON LAYOUT

CONCEPTUAL · NOT TO SCALE



many repeated execution lanes across many SMs

→ **Same budget, different priority:** CPUs spend more on fast, flexible execution; GPUs devote more silicon to repeated arithmetic throughput.

Many threads in flight keep memory and math busy.

The GPU exposes a much larger concurrency window—independent work can fill wide execution and memory pipelines.

CPU Smaller window same 32 operations · conceptual rounds →

	R1	R2	R3	R4	R5	R6	R7	R8
lane 0	01	05	09	13	17	21	25	29
lane 1	02	06	10	14	18	22	26	30
lane 2	03	07	11	15	19	23	27	31
lane 3	04	08	12	16	20	24	28	32

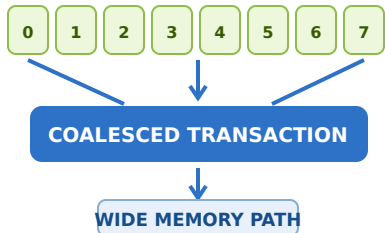
Modern CPUs also use multicore + SIMD, but across a smaller concurrency window.

GPU Massive window many resident threads ready to run



Hardware schedules resident warps onto finite lanes—launched threads are not all physically active at once.

ADJACENT THREAD ACCESSES

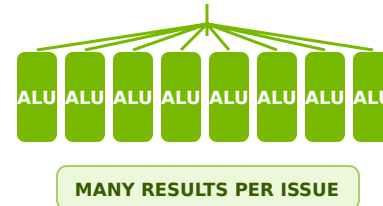


MEMORY THROUGHPUT

Keep bandwidth busy

Coalesced accesses make wide transactions efficient; many outstanding requests help sustain bandwidth.

WARP INSTRUCTION: FMA



COMPUTE THROUGHPUT

Keep math lanes busy

SIMT execution applies an instruction across many lanes, raising aggregate FLOP/s when work stays converged.

→ **Parallelism creates throughput** when access patterns are coalesced, enough independent work is available, and execution lanes stay useful.

When one warp waits, the GPU issues another.

Resident warp state stays on-chip, so the scheduler can select ready work with very little handoff overhead.

One SM · seven issue opportunities

IDEAL LATENCY-HIDING EXAMPLE

	t0	t1	t2	t3	t4	t5	t6
issued →	A	B	B	C	C	D	A
Warp A	LOAD	WAIT	WAIT	WAIT	WAIT	WAIT	USE DATA
Warp B	ready	MATH	MATH	—	—	—	—
Warp C	ready	ready	ready	MATH	MATH	—	—
Warp D	ready	ready	ready	ready	ready	MATH	—

Warp A still waits.

The memory latency is not shortened.

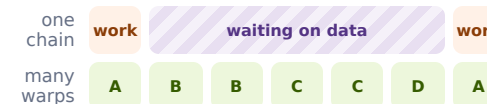
The issue path stays useful.

B, C, and D cover the gap with independent work.

What the scheduler does

- 1 A load or dependency stalls Warp A.
- 2 A's execution state remains **resident on-chip**.
- 3 The scheduler selects a **ready** Warp B, C, or D.
- 4 Warp A resumes when its data is ready.

Why multiple warps matter



- Needs enough independent, resident warps.
- Registers and shared memory limit residency.
- If every warp waits, the SM can still idle.

→ **Hide latency; do not eliminate it.** The GPU keeps meaningful work flowing by choosing another ready resident warp.